

구동 제어 가이드

자율주행 알고리즘의 출력 → Isaac Sim 제어 입력 연결 방법. 테스트 코드: Itutorial/car/06_extension_controllers.py

자율주행 알고리즘의 제어 출력

대부분의 자율주행 스택은 다음 중 하나를 출력합니다:

Type A: (steering_angle, desired_velocity)
→ Pure Pursuit, Stanley, RL policy 등

Type B: (steering_angle, acceleration, brake)
→ MPC, End-to-End 등

Type C: (steering_angle, throttle_percent)
→ 실차 ESC/VESC 직접 제어

Isaac Sim 제어 방법 6가지와 매핑

자율주행 → Isaac Sim 매핑표

알고리즘 출력	Isaac Sim 방법	적합도
desired_velocity →	③ write_joint_velocity	★★★ 가장 간편
desired_velocity →	① set_joint_velocity_target	★★ PD 응답 느림
acceleration →	② set_joint_effort_target	★★★ 가장 현실적
throttle_percent →	② effort (% × max_torque)	★★★★ 실차와 동일
steering_angle →	write_joint_position_to_sim	★★★★ 모든 경우

상세 연결

```
# 알고리즘 출력
steer_rad = pure_pursuit(...)    # rad
desired_vel = 3.0                # m/s

# Isaac Sim 연결
wheel_vel = desired_vel / wheel_radius    # m/s → rad/s 변환

# 방법 ③ (추천): 속도 강제 — 즉시 반응, 간편
robot.write_joint_velocity_to_sim(wheel_vel, joint_ids=drive_ids)
robot.write_joint_position_to_sim(steer_rad, joint_ids=steer_ids)
```

Type A: (steering, desired_velocity)

```
# 알고리즘 출력
steer_rad = mpc_output.steer    # rad
accel = mpc_output.acceleration # m/s²
brake = mpc_output.brake        # 0~1

# Isaac Sim 연결 — 방법 ② (추천): effort
#
```

```

# 물리:  $F = m \times a$ 
#  $\text{torque} = F \times \text{wheel\_radius}$ 
#
mass = 2.5 # kg (차량 질량)
if brake > 0:
    force = -brake * mass * 9.81 * 0.8 # 브레이크 마찰력
else:
    force = mass * accel # 구동력

torque_per_wheel = force * wheel_radius / num_drive_wheels

robot.set_joint_effort_target(torque_per_wheel, joint_ids=drive_ids)
robot.write_joint_position_to_sim(steer_rad, joint_ids=steer_ids)

```

Type B: (steering, acceleration, brake)

```

# 알고리즘 출력 (실차 VESC와 동일)
steer_rad = controller.steer # rad
throttle = controller.throttle # -1.0 ~ +1.0

# Isaac Sim 연결 — 방법 ② effort
#
# 실차: ESC가 throttle% → 모터 전류 → 토크
# 시뮬: throttle% × max_torque → effort
#
# 모터 파라미터 (KV, 기어비 등으로 계산)
max_torque_per_wheel = Kt * I_max * gear_ratio # Nm

torque = throttle * max_torque_per_wheel
robot.set_joint_effort_target(torque, joint_ids=drive_ids)
robot.write_joint_position_to_sim(steer_rad, joint_ids=steer_ids)

```

Type C: (steering, throttle_percent)

수식 정리

단위 변환

desired_velocity (m/s) → wheel_angular_vel (rad/s)

$$\omega = v / r$$

여기서 $r = \text{wheel_radius}$ (m)

acceleration (m/s²) → torque (Nm)

$$F = m \times a$$

$$\tau = F \times r / n_{\text{wheels}}$$

여기서 $m = \text{vehicle_mass}$ (kg)

$r = \text{wheel_radius}$ (m)

$n_{\text{wheels}} = \text{구동 바퀴 수 (RWD=2, 4WD=4)}$

throttle (%) → torque (Nm)

$$\tau = \text{throttle} \times \tau_{\text{max}}$$

$$\tau_{\text{max}} = Kt \times I_{\text{max}} \times \text{gear_ratio}$$

여기서 $Kt = 60 / (2\pi \times KV)$

PhysX 내부 토크 계산

실제 적용 토크 = $\text{stiffness} \times (\text{pos_target} - \text{pos})$
+ $\text{damping} \times (\text{vel_target} - \text{vel})$
+ effort_target

$\text{clamp}(-\text{effort_limit}, +\text{effort_limit})$

방법	stiffness	damping	누가 토크 계산?
① velocity target	0	30	PhysX PD (느림)
② effort	0	0	내가 직접 (현실적)
③ force write vel	-	-	무시 (강제 설정)
④ force write pos	-	-	무시 (강제 설정)

각 방법의 물리적 의미

① Velocity Target (set_joint_velocity_target)

"PD 컨트롤러가 이 속도를 유지하도록 토크를 알아서 계산해줘"

$$\tau = \text{damping} \times (\text{target_vel} - \text{current_vel})$$

target=30 rad/s, current=0 $\rightarrow \tau = 30 \times 30 = 900 \text{ Nm} \rightarrow \text{clamp to effort_limit}$
target=30 rad/s, current=29 $\rightarrow \tau = 30 \times 1 = 30 \text{ Nm} \rightarrow \text{미세 조정}$

장점: 속도 자동 유지

단점: PD 응답이 느림 (특히 f1tenth USD의 높은 내부 damping)

② Effort (set_joint_effort_target)

"이 토크를 바퀴에 직접 걸어라"

$$\tau = \text{effort_target} \text{ (내가 지정)}$$

effort=2.0 Nm $\rightarrow F = 2.0/0.0365 = 54.8 \text{ N}$
 $\rightarrow a = 54.8/2.5 = 21.9 \text{ m/s}^2$ (계속 가속!)

장점: 실차와 가장 유사 (모터 토크 직접 제어)

단점: 속도 제한을 코드에서 관리해야 함 (안 하면 폭주/전복)

③ Force Write Velocity (write_joint_velocity_to_sim)

"PD 무시하고 바퀴 속도를 이 값으로 강제 설정"

바퀴가 즉시 해당 속도로 회전 (무한 토크 가정)

장점: 즉시 반응, 간편

단점: 비현실적 (실차에서는 즉시 속도 변경 불가)

④ Force Write Position (write_joint_position_to_sim)

"바퀴 각도를 이 값으로 강제 설정"

매 step: $\text{angle} += \text{desired_vel} \times \text{dt}$

장점: 정밀 위치 제어

단점: 속도가 아닌 각도 관리 (실차와 안 맞음)

추천

목적	추천 방법
빠른 프로토타입 / RL 학습	→ ③ force write velocity
현실적 동역학 테스트	→ ② effort + 속도 리미터
실차 Sim2Real 전이	→ ② effort (모터 파라미터 매핑)
경로 추종 알고리즘 테스트	→ ③ force write velocity
MPC 테스트	→ ② effort (acceleration 입력)

실전 코드 패턴

```
# ——— RL / 프로토타입 (간편) ———
def apply_control_simple(robot, steer_rad, desired_vel_mps):
    """Type A: steering + velocity → force write"""
    wheel_vel = desired_vel_mps / WHEEL_RADIUS
    robot.write_joint_position_to_sim(steer_rad, joint_ids=steer_ids)
    robot.write_joint_velocity_to_sim(wheel_vel, joint_ids=drive_ids)

# ——— Sim2Real (현실적) ———
def apply_control_realistic(robot, steer_rad, accel_mps2, brake):
    """Type B: steering + accel/brake → effort"""
    if brake > 0:
        force = -brake * MASS * 9.81 * 0.8
    else:
        force = MASS * accel_mps2
    torque = force * WHEEL_RADIUS / N_DRIVE_WHEELS
    torque = max(-MAX_TORQUE, min(MAX_TORQUE, torque))

    robot.write_joint_position_to_sim(steer_rad, joint_ids=steer_ids)
    robot.set_joint_effort_target(torque, joint_ids=drive_ids)

# ——— VESC 매핑 (실차 동일) ———
def apply_control_vesc(robot, steer_rad, throttle_pct):
    """Type C: steering + throttle% → effort via motor model"""
    torque = throttle_pct * KT * I_MAX * GEAR_RATIO
    robot.write_joint_position_to_sim(steer_rad, joint_ids=steer_ids)
    robot.set_joint_effort_target(torque, joint_ids=drive_ids)
```

Isaac Sim 내장 컨트롤러 (Extension)

isaacsim.robot.wheeled_robots extension이 제공하는 컨트롤러들. 모든 컨트롤러의 최종 출력은 **조인트 속도(rad/s)**입니다.

전체 연결 구조

자율주행 스택	Extension 컨트롤러	조인트 명령
경로 계획: (start, goal)	→ QuinticPolynomial ↓	→ 경로 (x,y,yaw,v,a)
경로 추종: (경로, 현재 위치) → stanley_control() (목표 속도, 현재 속도) → pid_control()		→ steering_angle (rad) → speed_cmd
저수준 제어: (linear_vel, angular_vel)	→ DifferentialController .forward([v, ω])	→ [ω_left, ω_right] joint velocities
(goal_position, goal_orientation)	→ WheelBasePoseController .forward(start, goal)	→ [ω_left, ω_right] (내부에서 DiffCtrl 호출)
(steering_angle, speed)	→ Ackermann geometry (수동 계산)	→ [δ_left, δ_right, ω_wheels]

1. DifferentialController

역할: (선속도, 각속도) → (좌바퀴 속도, 우바퀴 속도)

⚠ 주의: wheel_radius와 wheel_base 단위가 cm!

생성:

```
DifferentialController(
    wheel_radius = 3.65,      # cm (= 0.0365 m)
    wheel_base = 20.0,       # cm (= 0.20 m)
)
```

호출:

```
actions = ctrl.forward([linear_vel, angular_vel])
# linear_vel: m/s (전진 속도)
# angular_vel: rad/s (회전 속도)
# → actions.joint_velocities = [ω_left, ω_right] (rad/s)
```

수식 (Unicycle 모델):

$$\omega_{\text{left}} = (2V - \omega b) / 2r$$

$$\omega_{\text{right}} = (2V + \omega b) / 2r$$

V = 선속도 (m/s)

ω = 각속도 (rad/s) (좌회전 양수)

b = 트랙 폭 (cm)

r = 바퀴 반지름 (cm)

자율주행 연결:

MPC 출력: (steering_angle, velocity)

↓

Bicycle → Unicycle 변환:

```
angular_vel = velocity × tan(steering_angle) / wheelbase
linear_vel = velocity
```

↓

DifferentialController.forward([linear_vel, angular_vel])

↓

$[\omega_{\text{left}}, \omega_{\text{right}}] \rightarrow \text{write_joint_velocity_to_sim}()$

2. WheelBasePoseController

역할: 목표 좌표 → 자동으로 이동 (DifferentialController를 내부 호출)

생성:

```
pose_ctrl = WheelBasePoseController(  
    open_loop_wheel_controller = diff_ctrl,  
    is_holonomic = False,  
)
```

호출:

```
actions = pose_ctrl.forward(  
    start_position = [x, y],          # 현재 위치 (m)  
    start_orientation = yaw,          # 현재 방향 (rad)  
    goal_position = [gx, gy],        # 목표 위치 (m)  
    goal_orientation = g_yaw,        # 목표 방향 (rad)  
    lateral_velocity = 0.5,          # 전진 속도 (m/s)  
    yaw_velocity = 1.0,              # 회전 속도 (rad/s)  
)  
# → actions.joint_velocities = [ $\omega_{\text{left}}$ ,  $\omega_{\text{right}}$ ]
```

내부 로직:

- heading_error = $\text{atan2}(\text{goal_y} - \text{pos_y}, \text{goal_x} - \text{pos_x}) - \text{current_yaw}$
- distance = $||\text{goal} - \text{pos}||$

```
if distance < position_tol:  
    stop  
elif |heading_error| > heading_tol:  
    제자리 회전 (linear=0, angular=yaw_velocity)  
else:  
    직진 (linear=lateral_velocity, angular=0)
```

→ `DifferentialController.forward([linear, angular])`

3. Quintic Polynomial Planner

역할: 시작 → 목표 사이의 부드러운 경로 생성

```
(time, x, y, yaw, v, a) = quintic_polynomials_planner(  
    sx, sy, syaw, sv, sa,          # 시작: 위치, 방향, 속도, 가속도  
    gx, gy, gyaw, gv, ga,          # 목표: //  
    max_accel, max_jerk, dt  
)
```

수식:

$$x(t) = a_0 + a_1 \cdot t + a_2 \cdot t^2 + a_3 \cdot t^3 + a_4 \cdot t^4 + a_5 \cdot t^5$$

6개 계수를 6개 경계조건으로 결정:

$x(0)=sx, \quad x'(0)=sv, \quad x''(0)=sa \quad (\text{시작})$
 $x(T)=gx, \quad x'(T)=gv, \quad x''(T)=ga \quad (\text{목표})$

4. Stanley Control + PID

역할: 경로 추종 조향각 계산 + 속도 제어

```
steering = stanley_control(state, cx, cy, cyaw, last_idx)
speed_cmd = pid_control(target_speed, current_speed)
```

Stanley 수식:

$$\delta = \text{heading_error} + \text{atan}(k \times \text{crosstrack_error} / (\text{speed} + \epsilon))$$

heading_error = path_yaw[nearest] - vehicle_yaw

crosstrack_error = 차량과 경로 사이 횡방향 거리

k = Stanley gain (보통 0.5)

PID 수식:

$$\text{speed_cmd} = K_p \times (\text{target_speed} - \text{current_speed})$$

5. Ackermann Geometry

역할: 가상 조향각 → 좌/우 바퀴의 실제 각도

입력: δ (가상 중앙 조향각)

출력: δ_{left} , δ_{right} (실제 좌/우 바퀴 각도)

```
R = L / tan(|δ|)           # 선회 반경
δ_left = atan(L / (R - d/2)) # 내륜 (더 많이 꺾임)
δ_right = atan(L / (R + d/2)) # 외륜 (덜 꺾임)
```

L = 축간거리 (wheelbase)

d = 트랙 폭 (track_width)

조인트에 실제로 전달되는 것

USD 조인트는 3가지 입력을 받습니다:

1. Position target (rad) — 목표 각도 (스티어링 조인트)
2. Velocity target (rad/s) — 목표 각속도 (구동 바퀴)
3. Effort target (Nm) — 직접 토크 (현실적 구동)

모든 Extension 컨트롤러의 최종 출력은 joint velocity (rad/s):

```
DifferentialController.forward() → joint_velocities [rad/s]
WheelBasePoseController.forward() → joint_velocities [rad/s]
Ackermann → wheel_angles [rad] + wheel_velocities [rad/s]
Stanley → steering_angle [rad]
PID → speed_command → ω = v/r [rad/s]
```

USD 조인트에 전달:

```
# 구동 (바퀴 회전)
robot.write_joint_velocity_to_sim(ω, joint_ids=drive_ids) # 강제 설정 (빠름)
robot.set_joint_velocity_target(ω, joint_ids=drive_ids) # PD 제어 (느림)
robot.set_joint_effort_target(τ, joint_ids=drive_ids) # 토크 직접 (현실적)

# 조향 (바퀴 방향)
robot.write_joint_position_to_sim(δ, joint_ids=steer_ids) # 각도 강제
robot.set_joint_position_target(δ, joint_ids=steer_ids) # PD로 각도 추종
```

각운동량이 아닌 각속도 또는 토크입니다. Isaac Sim에서 각운동량(angular momentum)을 직접 설정하는 API는 없습니다. 토크(Nm)가 가장 저수준이고, PhysX가 내부적으로 $L = I \times \omega$, $\tau = I \times \alpha$ 를 계산합니다.